

Mixed reality test environment for autonomous cars using Unity 3D and SUMO

1st Mátyás Szalai

*Dept. of Control for Transportation and Vehicle Systems
Budapest University of Technology and Economics
Budapest, Hungary
szalai.matyas@mail.bme.hu*

2nd Balázs Varga

*Dept. of Control for Transportation and Vehicle Systems
Budapest University of Technology and Economics
Budapest, Hungary
varga.balazs@mail.bme.hu*

3rd Tamás Tettamanti

*Dept. of Control for Transportation and Vehicle Systems
Budapest University of Technology and Economics
Budapest, Hungary
tettamanti@mail.bme.hu*

4th Viktor Tihanyi

*Dept. of Automotive Technologies
Budapest University of Technology and Economics
Budapest, Hungary
viktor.tihanyi@gjt.bme.hu*

Abstract—Developing a vehicle is always a long and complex task. This is especially true for autonomous cars. Tasks performed by the driver are taken over by the vehicle and must be performed with maximum reliability. Developing these systems is a difficult task, especially due to limited testing capabilities. Testing a vehicle in a closed environment is safe and controlled, but the variety of test scenarios is limited. By using mixed reality environments, one can create a diverse environment around a real test vehicle, with traffic, obstacles, and unexpected situations. The real movement of the test vehicle allows testing decision and motion planning level vehicular functions. Information from the virtual world can be considered as input to the vehicle sensor. Mixed reality or digital twin simulation environments can greatly assist the autonomous vehicle development process and also serve as a basis for validation procedures for such systems. This article introduces a mixed reality simulation environment that integrates a real test vehicle into a virtual environment, can handle other real and virtual obstacles, contains traffic simulations, and visualizes all of these.

Index Terms—ViL, SciL, test environment, traffic simulation, mixed reality, autonomous vehicles

I. INTRODUCTION

Nowadays, the automotive industry is increasingly focused on providing vehicles with self-driving capabilities [1]. The main goal is clearly a fully self-driving vehicle, but achieving this is a long process. Plenty of simulations, tests, validation procedures are still needed to enable for the society to use these future vehicles freely. These processes are not fully defined yet. Therefore, significant improvements are being made in many directions to make the development of autonomous vehicles as fast as possible.

An autonomous vehicle consists of several layers in architectural terms: Perception Layer, Decision Layer, Navigation Layer and Action Layer [2]. These layers require different types of testing procedures. The perception layer is responsible for processing the data measured by the vehicle's vision sensors (LiDaR, Radar). For verification, Hardver-in-the-Loop (HiL) tests can be used. The verification of the three other

layers require test drives or simulations. The decision layer generates the output signal for the action layer based on the perception layer's output. The action layer is responsible for controlling the actuators. The task of the navigation layer is the positioning and trajectory planning. Different simulation software are available to test these features. There are simulators with sensor models whose outputs can be used to test the operation of the decision layer (e.g. CARLA, rFpro, Vires VTD, AirSim, PreScan). Vehicle dynamics simulators can be used for testing trajectory planning algorithms and different control algorithms (e.g. CarMaker, CarSim). Traffic simulation software are available for testing movement behavior in traffic (e.g. SUMO, Vissim). However, fully autonomous behavior requires the combined operation of these functions, testing them individually does not adequately reflect real-world traffic scenarios. For this reason, the development of standalone simulation suites or co-simulation frameworks ([3], [4], [5], [6]) strives to be able to perform combined tasks, ensuring that complex systems can be tested.

Several co-simulation systems exist or are under development to test various automotive or traffic control functions. In [7], an emergency light function is implemented to illuminate obstacles that are difficult to see at night. Their solution is based on VIRES VTD and CAN communication with the test vehicle. In [8], a driver-centric simulator was presented, where the traffic is generated by microscopic models. This simulator is based on the connection between the microscopic traffic simulator SUMO [9] and the 3D graphic engine Unity3D through TraCI protocol. In [10] a solution is presented in which the DYNA4 vehicle dynamic simulator is connected to the SUMO microscopic traffic simulator. In [11] a connection between a vehicle dynamic simulator and a traffic simulator is being presented. The Co-Simulation of SUMO and IPG Carmaker is also useful for refining the traffic flow model based on vehicle dynamic simulations and for cruise control algorithms in a stochastic behavioral traffic

flow. [12] performs a concept how to model a Vehicle-Ad-Hoc-Network system in a 3D traffic environment. Unity 3D was used to generate Vehicles, Buildings, Road network, and the Vanet system. Some simulation tools provide a complete solution for co-simulations. CARLA [3] offers Autonomus Driving sensor suites, fast simulation for planning and control, maps generation, traffic scenarios simulations and ROS (Robot Operating System [13]) integration via ROS-bridge. PreScan [4] also provides great testing opportunities for Advanced Driver Assistance Systems (ADAS) with Model-in-the-Loop (MiL), Software-in-the-Loop (SiL) and (HiL) simulation environments.

Our solution is based on former research [14] and builds on SUMO microscopic traffic simulator and Unity 3D game engine. The communication interface between the simulation environment and the VUT (Vehicle Under Test) is implemented in Python. This interface also allows communication between the simulation environment and the test vehicle. The purpose of the solution we have chosen is to assist in the mixed reality testing of autonomous vehicles in a freely definable environment. With SUMO, we can generate a variety of traffic situations or real-world traffic flows, that will help testing autonomous vehicles in traffic. With Unity, we can customize the virtual test environment to suit our own needs and creating test cases that would be difficult to implement in reality. The game engine allows us much more flexibility in scenario generation compared to out of the box simulation software. All of these features are completed by the behavior of a real vehicle connected to the simulation in real-time, so-called Vehicle-in-the-Loop (ViL) feature. In virtual reality, this test vehicle will be cloned. We call this cloned virtual vehicle a Digital Twin. The system also provides the ability to use for Scenario-in-the-Loop (SciL) simulation.

This article describes the developed simulation system and its possible applications. Section II describes the simulation environment. Section III gives examples of the control options of the test vehicle. Section IV summarizes the simulation results. Finally, Section V concludes the paper.

II. THE SOLUTION OF THE TEST ENVIRONMENT

The main reason of the simulation system development was to enable a real test vehicle to be tested in a virtual environment. A ViL simulation environment requires real-time communication with the vehicle, which was made possible through the car's CAN network. An additional node was connected to the vehicle's CAN network that was responsible for the data exchange between the car and the simulation server wirelessly. After processing the information received from the vehicle, the digital twin of the test vehicle is defined. This digital twin mimics the motion of the test vehicle in the virtual environment. The virtual environment contains the following ingredients: virtual map, traffic, obstacles, pedestrians, traffic lights, traffic signals. We know the environment of the virtual twin, so sensor information can be fed back to the real vehicle relative to its motion. Receiving and transmitting CAN messages is solved in a Python environment. System components

such as SUMO and Unity 3D are also communicated through a TCP connection developed in the Python environment. A Choda device establishes a connection between the vehicle and the communication management system. The structure of the system is shown on Fig. 1.

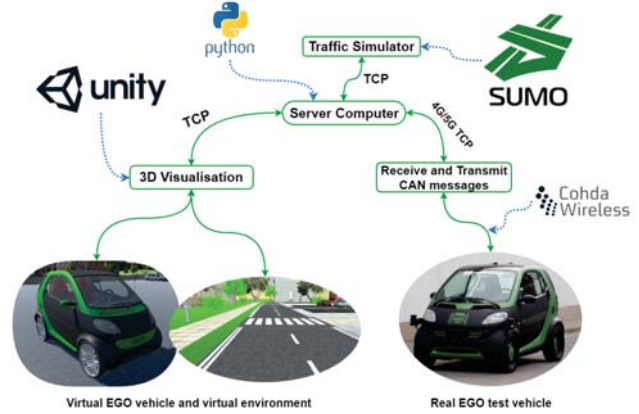


Fig. 1. The structure of the developed mixed reality system

A. Python server

The core of the system is a server developed in Python. This component is responsible for communication with the vehicle, visualization module and traffic simulation. This component is also responsible for synchronizing the different components of the test environment. The traffic scenario to be tested is translated into Python's arithmetic. Commands for virtual objects or the VUT are given based on the predefined scenario. For example, moving various obstacles (real or virtual) based on the world position of the VUT. Based on the obstacles calculated distance to the test vehicle, sensor information is generated and transmitted to the vehicle's CAN network and then to the computer controlling the test vehicle. Alternatively, the system can handle multiple real-world external objects, so you can place either real dummies or traffic objects in the virtual environment. They function in the same way as the digital twin of the car. During the visualization, every real object that is connected to the server computer moves according to reality, and its movement can even be set programmatically. This will expand the ViL simulation described above to a SciL simulation.

B. Unity 3D visualization

The main purpose of Unity 3D is visualization of the test. Displaying virtual simulations is important for evaluating simulations. In addition, the simulations provide a spectacular demonstration of some autonomous functions of the test vehicles.

1) *The creation of the Digital Twin:* The primary task was to create a digital twin for the test vehicle. This digital twin should move in the simulation environment in the same way as the real vehicle for which the real vehicle GPS device measurement data is used. These position data should be

accurate within 1 to 2 cm for smooth motion during simulation. For more inaccurate measurements, determining the vehicle's position requires some sensor fusion or estimation between measurement points (i.e. Simultaneous Localization And Mapping (SLAM)). The resulting position puts the vehicle on a virtual map, defined by GPS coordinates, with the same layout and boundaries as the test environment (as the virtual vehicle shall not go over the map borders, do not go through buildings in reality or in virtual reality). In addition to positioning the vehicle, the wheels shall be rotated and turned too. For this procedure, wheel speed sensor data, GPS sensor speed data, and steering angle sensor data were used. The vehicle is therefore moved not by the output torque and steer angle but by its instantaneous position and the heading.

2) *The creation of the Digital Environment:* There are several ways to create an environment. The method depends on whether the virtual testing environment is built on a real-world environment or an arbitrary scene. If we want to display the available area in digital form, we can create an environment based on GPS coordinates. In this case, the lateral and longitudinal coordinates shall be converted into x-y coordinate data using the SUMO traffic simulator's NETEDIT road network editor. So, this coordinate system defines the location of the digital environment based on the road network created in SUMO. The environment created in this way can be an exact replica of the real environment, but it can be expanded freely as long as it is not conflicting with real word obstacles. If we want to create a completely unique simulation environment, we only need to consider the geometric constraints of the real test environment.

3) *Handling moving objects:* In addition to the test vehicle and the constant environment, additional moving objects are required. These objects are vehicles created by the traffic simulator, virtual-generated dummy objects, or digital twins of real dummies. Due to the fact that the traffic simulator generated swarm vehicles shall be added or destroyed in runtime in accordance with the traffic simulator. Thus, a predefined Unity 3D prefab is cloned with all its properties. For vehicles, a new vehicle is randomly generated from the available vehicle list based on size. In the case of pedestrians, a clone is generated based on a predefined pedestrian prefab. These clones are dynamically managed by the system at runtime, reaching each objects in all steps one by one to reposition objects.

C. Sumo - Road Network and Traffic Generation

The purpose of SUMO traffic simulator [9] is generating the road network identical to its Unity 3D representation and create virtual traffic. SUMO is an open source traffic simulator with TraCi interface for easy to use Python environment. The NETEDIT plugin can be used to build a road network based on an OSM map or completely manually. OSM-based networks include lateral and longitudinal coordinates that can be transformed into a cartesian (x-y) coordinate system or vice versa. This coordinate system is matched in the two simulators, SUMO's and Unity's maps. The main task of the traffic simulator is to generate traffic. SUMO offers the

ability to generate traffic based on the characteristics of real traffic flows. We can route this traffic either through a fixed route or randomly over the entire network. We can define traffic lights and traffic light programs in the SUMO network. Information connecting to these issues can handle through the TraCi interface.

D. Data flow

These components together provide the simulation environment. The connection between them has already been described in Fig. 1. In addition to the relationship between the elements, it is important to determine the flow of information. The data transmitted to the visualization module is divided into 3 main sections. The first is in connection with the EGO vehicle, information about the real test vehicle. The second is information about other objects (pedestrians, vehicles). The third is information about traffic lights. Information for each part of the message is contained in TABLE I.

TABLE I
THE INFORMATION ABOUT THE THREE MESSAGE ELEMENTS

EGO object	Other Objects	Traffic Lights
ID (EGO)	ID	junction ID
pos_x	pos_x	lane index ID
pos_y	pos_y	pos_x
speed	speed	pos_y
heading	heading	state (o, r, y, g)
steering angle	brakelight state	
brakelight state	sizeclass	

Each object has an ID, an x and a y position. In the case of an EGO vehicle, the message also contains the speed of the vehicle, the angle of the vehicle, the steering angle, and the state of the brake light. For other objects, the steering angle is not sent because it is not included in SUMO. However, a variable called "sizeclass" is used. It refers to the size of the object associated with the ID. If sizeclass is set to 1, then it is a pedestrian, if it is 11, 12 or 13, it is a small, medium or large vehicle. The brake light message is always set to 0 for pedestrians. Junction ID refers to the intersection ID, lane index ID refers to the lanes at the intersection for traffic lights. The position in this case is currently unused. The status of the lamp per link is carried by the state variable. The status can be o - off, g - green, y - yellow and r - red.

The information from the EGO object passes through the information chain described above to visualization. Based on the information received from the vehicle's CAN network, the test vehicle must be placed within the Python server in the traffic simulation software so that it can be detected by virtual traffic. To do this, SUMO TraCi will convert its GPS coordinate and then position it within the network with its command "X, Y = traci.simulation.convertGeo (Longitudinal, Lateral, fromGeo = True)". The position and angle of all vehicles generated in SUMO are then retrieved. SUMO defines the position of vehicles as the center of its front bumper. Thus, for an EGO vehicle, the position must first be transformed from the center to the front bumper based on the known length.

For other vehicles the position from the front bumper must be transformed to the center, based on the length of the particular vehicle. Pedestrians defined in that scenario must also be placed in SUMO. For real pedestrian test dummies, it is positioned based on its measured position, and for virtual dummy, it is based on its predefined path. This outputs all the attributions of all participants needed to visualize them.

It is possible to design the simulation environment without a real test vehicle. In this case, we can directly control the virtual EGO vehicle that has been defined previously as a digital twin. This way, the data that was previously generated by the real VUT will be come from its digital representation. In this case, the TCP connection between the Python server and Unity 3D is bidirectional. The returned message contains the information needed to place the test vehicle on the traffic simulation: *pos_x*, *pos_y*, *heading*. Virtual sensor information can be generated too that the test environment can still be used for certain simulations.

E. Sensor simulation

In order for the test vehicle to detect its environment, it is also necessary to develop a sensing algorithm. To this end, the position, angle, length and width values queried from SUMO were used. Based on the known position of the EGO vehicle, a relevant nearest point can be calculated from all other objects, which can be defined as the distance sensed by the sensor. This point is called the relevant nearest point because it is not only based on the euclidean distance but also on the angle of travel of the EGO vehicle and the angle enclosed by that point. The resulting calculation method follows “Equation (1)”.

$$p_{weighted} = p_{distance} + |p_{angle} * weight_{angle}| \quad (1)$$

By weighting the angle between the vehicle and the sensed boundary point, it is possible to detect points in the direction of travel from the sensed points. This way, we can guarantee that the nearest points that are in the path of the vehicle are detected. Based on these, a distance value is calculated for each surrounding object. Depending on the number of sensors placed on the real test vehicle, the closest point in each detection zone is transmitted to the CAN bus of the vehicle.

F. Sensor simulation through Unity 3D

At this point, the simulation framework only uses ground truth virtual object positions based on the geometric boundaries (width, length) of each object. In addition, the communication with the visualization module is only one-way. As a future step, advanced sensor models can be made exploiting the capabilities of Unity 3D. In the testing of camera-based systems game engines are already being employed (e.g. Carla). Advanced sensor models for LiDAR and RADAR are yet the domain of advanced simulation suites such as PreScan. However, game engines have the flexibility to incorporate custom-made physics based sensor models. With this addition to the simulation environment, virtual sensor data could be generated from Unity 3D.

III. CONTROL OPTIONS FOR TESTING

The simulation system offers the possibility test it in different ways. The main task of the system is to implement a ViL and SciL system. In addition, the structure of the system allows simulations to be performed without the use of a real test vehicle, using only additional elements of the system. In this case, the EGO vehicle can be controlled in virtual reality or on the basis of signals generated by some other external algorithm, simulation.

A. Vehicle-in-the-Loop with CAN interface

In ViL simulations input signals are directly taken from the vehicle’s CAN network. The theoretical background for this is explained in detail in “Section II-B1”. The flow of information was described in “Section II-D”. In the simulation system, the flow of information completes a complete loop. The starting point is the CAN network of the vehicle from which the information is processed in the Python server, from there to the traffic simulator and to the visualization module, while calculating the data containing the false sensor information. These are fed back into the vehicle’s CAN network. These sensor data are structured in a format consistent with the data structure of the real sensors placed on the vehicle, eliminating the need for reprogramming of the vehicle control software. The CAN messages sent differ only in their IDs from the CAN messages of the real sensors, so they can be distinguished and taken into consideration at the same time.

B. Keyboard, Steering wheel, Play Seat

Another simple, little-altered control option is to control the digital test vehicle using a keyboard or a gaming steering wheel. In this case, the simulation input information comes from the Digital Test Vehicle. The position and angle in the digital world are sent to the server computer via the TCP connection. Based on these, the test vehicle is placed in the traffic simulator. The handling of other objects is completely the same as before.

Vehicle control is accomplished using the *Horizontal* and *Vertical* axes defined in Unity. These axes are defined as inputs that are interpreted at different peripherals. For keyboards, the *Arrow* keys and the *WASD* keys are defined as these axes. Axes can have values between *-1* and *1*. Values also have a rise time even when the button is pressed, so the value will not be maximized immediately. For steering wheel controller, Play Seat, steering wheel movement, and throttle and brake pedal outputs also provide *Horizontal* and *Vertical* axis values. The displacement relative to the vehicle’s current position is determined by the value of the *Vertical* axis multiplied by the maximum speed set. The vehicle angle is determined by the values of the *Horizontal* axis and the maximum speed of rotation.

With this control, the interaction between the test vehicle and the environment can be tested. The sensor information is still computed and can be saved for later offline use or used online for testing an algorithm.

C. Autonomous driving algorithms with steering angle and torque

The result of the control process described in "Section III-B" is that the vehicle movement is not based on any vehicle model. Movement is not always realistic. This is not a problem with ViL simulations because it takes the motion dynamics of the digital twin from the real test vehicle. Modeling vehicle control generally requires an input acceleration and deceleration torque as well as an input steering angle. The lack of these limits the testing of some algorithms. To overcome this problem, you need to use the control available in the "Standard Asset", which is available for free in Unity. This simple asset is not as detailed as for example in IPG CarMaker. E.g. the tire model in Unity 3D's wheel collider object is only based on two splines. Unity 3D has some vehicle dynamics, not as good as other dedicated tools, it is possible to build custom vehicle dynamics models that are nearly as good as in the professional tools. The standard/built-in control algorithm also works with horizontal and vertical axis values, but generates torque-like values from them. This torque serves as the input for the predefined vehicle model. Thus, the vehicle movement is realistic and has dynamic characteristics. As a result, a control signal calculated by an algorithm would already be able to properly control the virtual test vehicle. Virtual sensor information is still available to the algorithm as input information.

IV. SIMULATION AND RESULTS

In the proposed test environment a vast variety of tests can be performed. Some tests were carried out to check the functionality of the system and developing functions, while others led to real tests through specific simulation cases. These experiences and results prove the legitimacy of the system created and point to its further development.

A. Manually controlled vehicle in virtual traffic

In order to test the simulation system with a real test vehicle, it was necessary to test the operation of the system. This was obviously aimed at eliminating unexpected bugs that would have been time-consuming to correct in real tests. Therefore, we first put a manual (in our case keyboard controlled) vehicle into the virtual traffic. This way we were able to test whether the logical architecture of the system was correct. A vehicle fitted with virtual traffic and controlled from the outside in terms of traffic is visible to other vehicles. The behavior of the test vehicle influences the flow of traffic and is present as a full participant. The presence of an inserted vehicle due to its incompletely realistic behavior may in some cases cause an unexpected reaction. In many cases, the approaching vehicles stopped with emergency braking. However, this is acceptable at this stage of development as we have repeatedly created placements that would not occur in traffic.

B. Real tests at ZalaZone test track

With the help of the created simulation environment real tests were performed in the SmartCity zone of the Zalaegerszeg ZalaZone test track [15]. During these tests, a

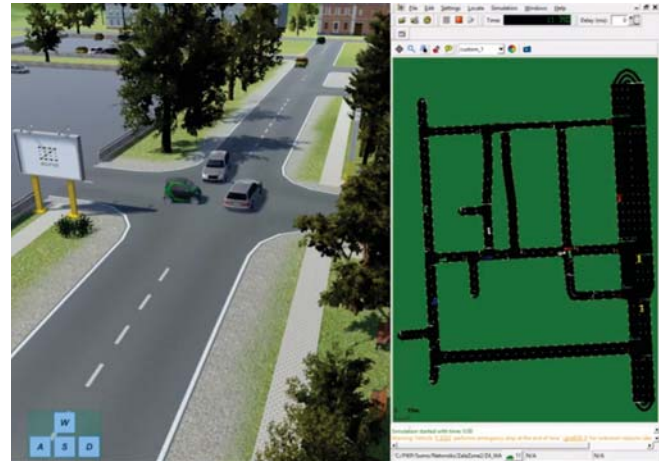


Fig. 2. Testing with keyboard controlled vehicle

SciL scenario was demonstrated that included a real dummy pedestrian, a virtual vehicle and a virtual pedestrian in addition to the real test vehicle.

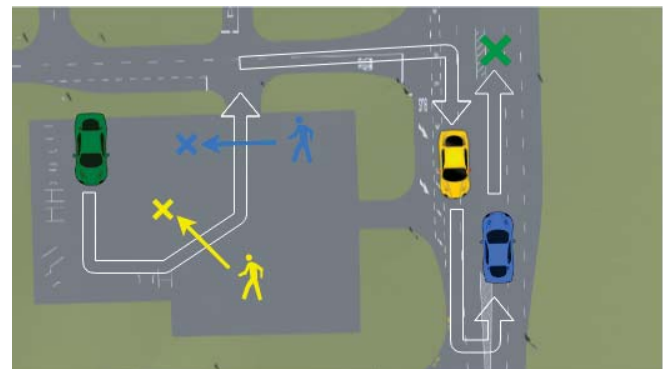


Fig. 3. The schematic view of the Scenario in the ZalaZone SmartCity

The test shown is described in Fig. 3. The test vehicle started from the location of the green vehicle icon and followed the white arrows. For the first task, his path was crossed by a virtual pedestrian (yellow). The pedestrian crossed the test vehicle in such a way that it had to interrupt his journey by braking. In the second task, a real dummy pedestrian (blue) crossed the test vehicle's path, and interrupts its movement. This dummy pedestrian is also displayed in the virtual environment (i.e. has a digital twin). At the next junction, the driver took control of the test vehicle, followed the path and stopping at an appointed position on the multi-lane road. The test vehicle then continued in self-driving mode with adaptive speed control relative to a virtual vehicle (yellow) in front. After that, the driver turning round, and a traffic jam assist functions was demonstrated with another real test vehicle (blue).

The relationship between the elements was stable and fast enough. The movement of the vehicle in the virtual space was completely realistic, the movement was practically in real time. The simulation frequency can be set between 10 and

20 ms. And with this frequency, it is fully guaranteed that, at low speeds, all the information gets to the test environment in time and back to the vehicle. Based on the experience of the simulation, the framework can be used to perform tests for certain functions of self-driving vehicles.

In our case, the main result was the detection of virtual objects. In case of virtual pedestrian intervention, an emergency brake function was implemented and the adaptive cruise control function was successfully executed with the virtual vehicle.

C. VR opportunities

Due to the design of the existing system, additional visualization units can be connected to the system. The main task of the existing visualization model is to ensure the traceability of the simulation. However, Unity offers the ability to view or intervene the virtual reality through a VR device.

Expansion with such a VR system can be easily accomplished by receiving the message being sent as a second client and displaying the environment on a VR device. Unity has support for Android devices, so any android device with a gyroscope (above Android API level 21) can be joined in the simulation. Supported devices include Google Cardboard, Oculus, Daydream, Gear VR, HTC VIVE, Microsoft HoloLens, Playstation VR, Windows 10 [16].

V. CONCLUSION AND FUTURE WORK

The original aim of testing a real vehicle in a virtual environment has been achieved. In addition, it can be stated that the created simulation system has further potentials. The use of traffic simulation significantly increases the number of possible scenarios. Thus, a test vehicle with its full control algorithm can be tested in multiple traffic situations, which in reality is not possible due to strict regulation. Traffic lights can be used to build a realistic urban transport system. By allowing the system to handle multiple real objects at the same time, more complex scenarios can be realized. Such a system can assist both development processes and validation procedures. If a vehicle is able to make the right decisions and interventions based on virtual reality, we can make statements about the decision, navigation and the action layer. In case of a SciL scenario with multiple real dummy objects, the perception layer can also be tested.

However, in order to actually use such a system for validation procedures, further improvements are needed. It is important that both the communication [17] and the scenario description [18] follow the format of their respective standards. One needs to find the right channel for communicating with vehicles developed by different OEMs and Tier 1 - 2 suppliers. For our tests, we used our own test vehicle from the Budapest University of Technology and Economics, which has a CAN network and has full information about the vehicle. Obviously, this extent of customizability is not always available in vehicles developed for the market. Future work will integrate these standards into the system. Another improvement is the improvement of the perception of the virtual environment.

Aligning these and making the virtual sensor information available for the test vehicle remain a challenge.

ACKNOWLEDGMENT

This project has been supported by the European Union, co-financed by the European Social Fund. EFOP-3.6.2-16-2017-00002

REFERENCES

- [1] T. Derenda, M. Zanne, M. Zöldy, and A. Torok, "Automatization in road transport: a review," *Production Engineering Archives*, vol. 20, pp. 3–7, 09 2018.
- [2] W. Huang, K. Wang, Y. Lv, and F. Zhu, "Autonomous vehicles testing methods review," in *2016 IEEE 19th International Conference on Intelligent Transportation Systems (ITSC)*, Nov 2016, pp. 163–168.
- [3] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun, "CARLA: an open urban driving simulator," *CoRR*, vol. abs/1711.03938, 2017. [Online]. Available: <http://arxiv.org/abs/1711.03938>
- [4] Tass. (2019, Oct.) Prescan - simulation of adas & active safety (2019.2). [Online]. Available: <https://tass.plm.automation.siemens.com/prescan>
- [5] Vires. (2019, Oct.) VTD - Virtual Test Drive. [Online]. Available: <https://vires.com/vtd-vires-virtual-test-drive/>
- [6] Mathworks. (2019, Oct.) Matlab - Automated Driving Toolbox. [Online]. Available: <https://www.mathworks.com/products/automated-driving.html>
- [7] Y. Laschinsky, K. von Neumann-Cosel, M. Gonter, C. Wegwerth, R. Dubitzky, and A. Knoll, "Evaluation of an active safety light using virtual test drive within vehicle in the loop," in *2010 IEEE International Conference on Industrial Technology*, March 2010, pp. 1119–1122.
- [8] C. Biurrun-Quel, L. Serrano-Arriazu, and C. Olaverri-Monreal, "Microscopic driver-centric simulator: Linking Unity3D and SUMO," in *Recent Advances in Information Systems and Technologies*, Á. Rocha, A. M. Correia, H. Adeli, L. P. Reis, and S. Costanzo, Eds. Cham: Springer International Publishing, 2017, pp. 851–860.
- [9] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, "Microscopic traffic simulation using sumo," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, Nov 2018, pp. 2575–2582.
- [10] J. Kath, B. Schott, and F. Chucholowski, "Virtual testing by coupling high fidelity vehicle simulation with microscopic traffic flow simulation," 06 2018.
- [11] J. Kath and S. Krause, "Integrated simulation of microscopic traffic flow and vehicle dynamics," in *IPG Apply & Innovate*, 09 2016.
- [12] F. Michaeler and C. Olaverri-Monreal, "3d driving simulator with vanet capabilities to assess cooperative systems: 3dsimvanet," in *2017 IEEE Intelligent Vehicles Symposium (IV)*, June 2017, pp. 999–1004.
- [13] (2019, Dec.) ROS.org. [Online]. Available: <http://wiki.ros.org/Documentation>
- [14] T. Tettamanti, M. Szalai, S. Vass, and V. Tihanyi, "Vehicle-In-the-Loop test environment for autonomous driving with microscopic traffic simulation," in *2018 IEEE International Conference on Vehicular Electronics and Safety (ICVES 2018)*, Madrid, Spain, 12–14. Sept. 2018, pp. 295–300.
- [15] M. Horváth, Q. Lu, T. Tettamanti, A. Török, and Z. Szalay, "Vehicle-In-The-Loop (VIL) and Scenario-In-The-Loop (SCIL) automotive simulation concepts from the perspectives of traffic simulation and traffic control," *Transport and Telecommunication Journal*, vol. 20, pp. 153–161, 04 2019.
- [16] (2019, Oct.) Unity user manual (2019.2) - xr. [Online]. Available: <https://docs.unity3d.com/Manual/XR.html>
- [17] A. Dua, N. Kumar, and S. Bawa, "A systematic review on routing protocols for vehicular ad hoc networks," *Vehicular Communications*, vol. 1, p. 33–52, 01 2014.
- [18] T. Thomsen and P. R. Mai, "Asam openscenario - list of features and requirements," Jan. 2019.